INF560 / Unidad 3 / GL09

GUÍA DE LABORATORIO

Autenticación de usuarios

Registro, login y logout; proteger rutas y dar dueño a los eventos

Laravel 13 · Auth · Sesiones · PostgreSQL 18 · Proyecto acumulativo eventos-app · Duración 4 h

Métodos HTTP de la serie: GET POST PUT DELETE

Universidad Autónoma Tomás Frías · Facultad de Ciencias Puras · Ingeniería Informática

00 Antes de empezar

El `eventos-app` ya tiene un CRUD completo con validación y relaciones, pero todo es **público**: cualquier visitante puede crear, editar o eliminar eventos. En esta guía agregamos **autenticación**: los usuarios podrán registrarse e iniciar sesión, protegeremos las rutas de gestión y cada evento quedará asociado a su **creador**.

> **¿POR QUÉ NO USAMOS UN STARTER KIT?**
> Desde Laravel 12, `Breeze` y `Jetstream` están en modo solo-mantenimiento (no reciben nuevas funciones), y los nuevos starter kits oficiales apuntan a React, Vue o Livewire, no a Blade puro. Además, `Breeze` todavía trae Tailwind v3, mientras nuestro proyecto usa **Tailwind v4**. Por eso construiremos la autenticación **a mano** con las herramientas integradas de Laravel: más compatible con nuestro stack y, sobre todo, se entiende cómo funciona por dentro.

Objetivos de la guía

- Implementar registro, inicio y cierre de sesión con el Auth de Laravel.
- Proteger las rutas de gestión con el middleware `auth`.
- Mostrar enlaces distintos según haya sesión con `@auth` y `@guest`.
- Asociar cada evento a su usuario creador con una relación `user_id`.

Requisitos previos

- La GL08 completa. La tabla `users` ya existe (se migró en la GL05) y el modelo `User` viene con Laravel.

> **ALCANCE DE ESTA GUÍA**
> Cubrimos **autenticación** (quién eres). La **autorización** (quién puede editar qué evento) es la **GL10**. El restablecimiento de contraseña y la verificación por correo quedan fuera de esta guía.

01 Teoría breve: autenticación en Laravel

1.1 Autenticación vs. autorización

Autenticar es confirmar **quién** es el usuario (con email y contraseña). **Autorizar** es decidir **qué** puede hacer una vez identificado. Esta guía trata la primera; la segunda es la GL10.

1.2 Las piezas que ya tenemos

- La tabla `users` (`name`, `email`, `password`) y el modelo `App\Models\User`, incluidos en Laravel.
- El **guard web** basado en sesiones, ya configurado.

1.3 La fachada Auth y el hash

> **DEFINICIÓN — AUTH Y HASHING**
> La fachada `Auth` gestiona la sesión: `Auth::attempt()` verifica credenciales, `Auth::user()` da el usuario actual, `Auth::logout()` cierra la sesión. Las contraseñas **nunca** se guardan en texto plano: se almacenan con **hash** (bcrypt). En el modelo `User`, el cast `'password' => 'hashed'` las encripta automáticamente al guardarlas.

02 Ejercicio 1 — Registro de usuarios

Paso 1 — Controlador y rutas

terminal Copiar

```
$ php artisan make:controller Auth/RegistroController
```

routes/web.php Copiar

```
use App\Http\Controllers\Auth\RegistroController;

Route::get('registro',  [RegistroController::class, 'create'])->name('registro');
Route::post('registro', [RegistroController::class, 'store']);
```

Paso 2 — El controlador

app/Http/Controllers/Auth/RegistroController.php Copiar

```
<?php

namespace App\Http\Controllers\Auth;

use App\Http\Controllers\Controller;
use App\Models\User;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;

class RegistroController extends Controller
{
    public function create()
    {
        return view('auth.registro');
    }

    public function store(Request $request)
    {
        $datos = $request->validate([
            'name'     => ['required', 'string', 'max:255'],
            'email'    => ['required', 'email', 'unique:users,email'],
            'password' => ['required', 'confirmed', 'min:8'],
        ]);

        $user = User::create($datos);   // el cast 'hashed' encripta la contraseña

        Auth::login($user);             // inicia sesión automáticamente

        return redirect()->route('eventos.index')
                         ->with('exito', '¡Bienvenido, ' . $user->name . '!');
    }
}
```

- `confirmed` exige un segundo campo `password_confirmation` idéntico.
- `unique:users,email` impide registrar dos veces el mismo correo.
- `Auth::login($user)` deja al usuario con la sesión iniciada tras registrarse.

Paso 3 — La vista del formulario

resources/views/auth/registro.blade.php Copiar

```
<x-layout>
  <form action="{{ route('registro') }}" method="POST" class="max-w-sm space-y-4">
      @csrf
```

```
      <input type="text" name="name" placeholder="Nombre" value="{{ old('name') }}"
             class="w-full rounded-lg border-gray-300">
      <input type="email" name="email" placeholder="Correo" value="{{ old('email') }}"
             class="w-full rounded-lg border-gray-300">
      <input type="password" name="password" placeholder="Contraseña"
             class="w-full rounded-lg border-gray-300">
      <input type="password" name="password_confirmation" placeholder="Repetir contraseña"
             class="w-full rounded-lg border-gray-300">
      @error('email') <p class="text-sm text-red-600">{{ $message }}</p> @enderror

      <button class="rounded-lg bg-blue-600 px-4 py-2 text-white">Crear cuenta</button>
  </form>
</x-layout>
```

03 Ejercicio 2 — Inicio y cierre de sesión

Paso 1 — Rutas

routes/web.php Copiar

```
use App\Http\Controllers\Auth\LoginController;

Route::get('login',  [LoginController::class, 'create'])->name('login');
Route::post('login', [LoginController::class, 'store']);
Route::post('logout', [LoginController::class, 'destroy'])->name('logout');
```

> **NOTA**
>
> La ruta de login **debe** llamarse `login`: cuando un visitante sin sesión intente entrar a una ruta protegida, Laravel lo redirige a la ruta con ese nombre.

Paso 2 — El controlador

app/Http/Controllers/Auth/LoginController.php Copiar

```
public function create()
{
    return view('auth.login');
}

public function store(Request $request)
{
    $credenciales = $request->validate([
        'email'    => ['required', 'email'],
        'password' => ['required'],
    ]);

    if (! Auth::attempt($credenciales, $request->boolean('remember'))) {
        return back()->withErrors([
            'email' => 'Las credenciales no coinciden con nuestros registros.',
        ])->onlyInput('email');
    }

    $request->session()->regenerate();   // seguridad: renueva el id de sesión

    return redirect()->intended(route('eventos.index'));
}
```

```
public function destroy(Request $request)
{
    Auth::logout();
    $request->session()->invalidate();
    $request->session()->regenerateToken();

    return redirect()->route('eventos.index');
}
```

- `Auth::attempt($credenciales)` compara la contraseña con su hash; devuelve `true` si coincide e inicia la sesión.
- `session()->regenerate()` cambia el identificador de sesión al entrar, para prevenir el secuestro de sesión.
- `intended(...)` regresa a la página que el usuario quería antes de ser enviado al login.

> **EL LOGOUT VA POR POST, NO POR ENLACE**
>
> Cerrar sesión **modifica el estado**, así que se hace con un formulario `POST` y `@csrf` (nunca con un `<a>`). Lo agregaremos a la barra de navegación en el Ejercicio 4.

04 Ejercicio 3 — Proteger las rutas de gestión

Ver la lista y el detalle deben seguir siendo públicos, pero crear, editar y eliminar solo para usuarios con sesión. El middleware `auth` se encarga de ello.

routes/web.php Copiar

```
// Públicas: cualquiera puede ver
Route::get('eventos', [EventoController::class, 'index'])->name('eventos.index');
Route::get('eventos/{evento}', [EventoController::class, 'show'])->name('eventos.show');

// Protegidas: requieren sesión iniciada
Route::middleware('auth')->group(function () {
    Route::resource('eventos', EventoController::class)
         ->except(['index', 'show']);
});
```

Ahora `create`, `store`, `edit`, `update` y `destroy` quedan tras `auth`. Un visitante sin sesión que intente `/eventos/create` será enviado automáticamente al login.

> **NOTA**
>
> Como alternativa, en Laravel 11+ puede aplicar el middleware **dentro del controlador** implementando la interfaz `HasMiddleware` y su método estático `middleware()`. El enfoque por rutas de arriba es igual de válido y más visible.

05 Ejercicio 4 — Navegación según la sesión

En el layout, muestre enlaces de ingreso/registro a los invitados, y el nombre del usuario con un botón de salir a quienes tienen sesión. Las directivas `@auth` y `@guest` lo resuelven.

resources/views/components/layout.blade.php Copiar

```
<nav class="flex items-center gap-4">
    <a href="{{ route('eventos.index') }}">Eventos</a>
```

```
    @auth
        <span class="text-sm text-gray-600">Hola, {{ auth()->user()->name }}</span>
        <form action="{{ route('logout') }}" method="POST">
            @csrf
            <button class="text-sm text-red-600">Salir</button>
        </form>
    @else
        <a href="{{ route('login') }}">Ingresar</a>
        <a href="{{ route('registro') }}">Registrarse</a>
    @endauth
</nav>
```

Todo lo que esté entre `@auth` y `@else` solo se muestra con sesión iniciada; lo que sigue hasta `@endauth`, solo a invitados. `auth()->user()` es el usuario actual.

06 Ejercicio 5 — Cada evento con su creador

Paso 1 — Columna user_id en eventos

terminal Copiar

```
$ php artisan make:migration add_user_id_to_eventos_table
```

migration Copiar

```
Schema::table('eventos', function (Blueprint $table) {
    $table->foreignId('user_id')->nullable()->constrained()->nullOnDelete();
});
```

Paso 2 — La relación en ambos modelos

app/Models/Evento.php Copiar

```
public function usuario()
{
    // el nombre del método (usuario) no coincide con la columna (user_id),
    // por eso indicamos la clave foránea explícitamente
    return $this->belongsTo(User::class, 'user_id');
}
```

app/Models/User.php Copiar

```
public function eventos()
{
    return $this->hasMany(Evento::class);
}
```

Paso 3 — Asignar el creador al guardar

En `store()`, cree el evento **a través del usuario autenticado**: Laravel pone el `user_id` solo.

app/Http/Controllers/EventoController.php Copiar

```
public function store(EventoRequest $request)
{
    // crea el evento ya asociado al usuario en sesión
    $evento = $request->user()->eventos()->create($request->validated());

    $evento->etiquetas()->sync($request->input('etiquetas', []));

    return redirect()->route('eventos.index')
```

```
                    ->with('exito', 'Evento creado correctamente.');
}
```

Paso 4 — Mostrar el creador

resources/views/eventos/show.blade.php Copiar

```
<p class="text-sm text-gray-500">
    Creado por {{ $evento->usuario?->name ?? 'Desconocido' }}
</p>
```

LO QUE ACABA DE LOGRAR

Los usuarios se registran e inician sesión; solo ellos gestionan eventos; y cada evento sabe quién lo creó. Eso prepara el terreno para la **GL10**, donde usaremos ese `user_id` para autorizar: que **solo el dueño** (o un administrador) pueda editar o eliminar su evento.

07 Trabajo independiente

Consigna

- Cree una página `/mis-eventos` (protegida) que liste solo los eventos del usuario en sesión: `$request->user()->eventos()->latest()->paginate(9)`.
- Agregue su enlace en la barra de navegación, visible solo con `@auth`.
- Proteja también la gestión de `categorias` con el middleware `auth`.

Entregables

- Commits atómicos y push a `INF560-2026-02-Apellido`.
- Capturas: registro, login, navegación con sesión y la página «mis eventos».

08 Checklist de verificación

- ☐ El registro crea el usuario, encripta la contraseña e inicia sesión.
- ☐ El login usa Auth::attempt y regenera la sesión; el logout va por POST con @csrf.
- ☐ La ruta de login se llama 'login' y recibe a los no autenticados.
- ☐ create/store/edit/update/destroy están tras el middleware auth; index y show son públicas.
- ☐ La barra de navegación cambia con @auth/@guest y muestra el nombre del usuario.
- ☐ eventos tiene user_id y la relación Evento↔User está definida en ambos lados.
- ☐ Al crear un evento se guarda el user_id del usuario en sesión.
- ☐ Trabajo independiente: página «mis eventos» y categorias protegidas, en GitHub.

09 Problemas frecuentes

Síntoma	Causa y solución
`Route [login] not defined`	La ruta de login no existe o no se llama 'login'. Nómbrela con ->name('login').
`Auth::attempt siempre es false`	El email no existe, la contraseña es incorrecta, o se guardó sin hash (revise el cast 'hashed' del modelo User).

Síntoma	Causa y solución
`419 al cerrar sesión`	Falta @csrf en el formulario de logout, o se intentó con un enlace en vez de POST.
Bucle de redirección al login	Protegió también la propia ruta de login. Déjela fuera del grupo con middleware auth.
`SQLSTATE ... user_id null`	Creó el evento sin usuario. Use $request->user()->eventos()->create(...).
`Target class [Auth\...] does not exist`	El namespace del controlador no coincide con su carpeta (App\Http\Controllers\Auth).

10 Comandos y helpers de referencia

autenticación en Laravel Copiar

```
# La fachada Auth
Auth::attempt(['email' => ..., 'password' => ...])   # verifica e inicia sesión
Auth::login($user)                                    # inicia sesión con un modelo
Auth::user()   /   auth()->user()                     # usuario actual
Auth::check()                                         # ¿hay sesión?
Auth::id()                                            # id del usuario actual
Auth::logout()                                        # cierra la sesión

# En una petición
$request->user()                 # usuario autenticado
$request->session()->regenerate()   # renueva la sesión (tras login)
$request->session()->invalidate()   # destruye la sesión (en logout)

# Directivas Blade
@auth ... @else ... @endauth     # según haya sesión
@guest ... @endguest             # solo invitados

# Rutas
->middleware('auth')             # exige sesión iniciada
```

11 Rúbrica de evaluación

Criterio	Pts
Registro funcional con validación y hash de contraseña	20
Login con Auth::attempt y regeneración de sesión	15
Logout por POST con @csrf	10
Rutas de gestión protegidas con middleware auth	20
Navegación condicional con @auth/@guest	10
Relación evento↔usuario y user_id asignado al crear	15
Trabajo independiente: «mis eventos» y categorias protegidas	10
Total	100